Backend Developer Home Assignment - Buyer Throttling Service

Backend Developer Home Assignment @ Rise

Please read the instructions carefully. Expected effort is about **1–2 days**. We value depth and clear reasoning over breadth.

Main Objective

Build a **Buyer Throttling System**: a system that monitors the health of external buyers in a real-time auction platform and controls how much traffic each buyer receives.

Background

Our platform runs real-time auctions. For every ad request we call many external buyers at the same time, collect their bids, and pick a winner, all within about 150ms.

Some buyers degrade over time: high error rates, timeouts, outages. A bad buyer slows down every auction it takes part in. But buyers are also our revenue, so cutting a buyer off completely and forever is expensive.

Your system decides how much traffic each buyer should receive, based on its recent health.

Constraints you must design for:

- The auction service runs as **many pods at the same time**. Each pod makes its own per-request decisions, but all pods must converge on the same view of buyer health.
- The per-request check ("should I call this buyer?") is on the **critical path** of every auction. It must add as little latency as possible.
- The system handles very high traffic (think 100k+ requests per second across all pods).

Product Definition

Throttle rate

Each buyer has a `throttle_rate`, a number between 0.0 and the block level:

- `0.0` means fully open: every request is sent to the buyer.
- `0.95` (the default block level) means 95% of requests to that buyer are skipped.

Traffic to a buyer is never cut to zero, so we always keep receiving a signal about its health.

The Evaluator

A component that runs **periodically** (for example, every minute). For each buyer it:

1. Reads the buyer's aggregated stats for the last window (total requests, failed, timeouts), collected from all pods.
2. Decides the buyer's new `throttle_rate` based on those stats and its previous state.
3. Writes the new throttle state where the auction pods can read it.

Rules:

- If `failed / total` or `timeouts / total` reaches its threshold, the buyer is blocked: `throttle_rate` goes straight to the block level.
- If the window has fewer requests than the minimum sample size, the rates are not meaningful and must not trigger a block.
- After a block, every clean window lowers `throttle_rate` by the step size, until it reaches 0.0.
- A violation at any point, including in the middle of recovery, sends `throttle_rate` straight back to the block level.

Default configuration (all values must be configurable):

Parameter	Default
Failure rate threshold	0.7
Timeout rate threshold	0.3
Minimum sample size per window	100 requests
Block level	0.95
Recovery step size	0.1
Evaluation window	1 minute

The Auction-Side Throttling Component

A component used inside every auction pod. It exposes:

- `ShouldCall(buyerID) bool`: called before every buyer call. Decides whether to send this request, according to the buyer's current `throttle_rate`.
- `Record(buyerID, result)`: called after every buyer call, with the result `ok`, `failed`, or `timeout`.

How this component gets the throttle state and how it reports its stats is up to you. Keep the constraints above in mind.

Ambiguity

Some cases are deliberately not specified. For example: should a window below the minimum sample size count as "clean" during recovery? Where the rules don't say, **make a decision and document it** in your design doc.

Deliverables

Deliverable	Details
Design doc	A `DESIGN.md` in the repo (about 1 page): the components and how they interact, how stats and throttle state flow between them, how you kept the per-request check fast, how multiple pods stay consistent, and the trade-offs you made.
Code	Well-structured, with clear separation of concerns and meaningful interfaces.
Tests	Cover the evaluator rules (block, recovery steps, snap-back, minimum sample) and the edge cases you identified.
Store	Use any store you think fits this use case.
Runnable demo	`docker-compose up` should start the store, the evaluator, at least **2 auction-pod instances**, and a simple **traffic simulator** with fake buyers whose failure/timeout rates are configurable. We should be able to watch a bad buyer get blocked and then gradually recover (logs are enough).
README	How to run it, how to configure it, and how to see the throttling in action.

You can write it in any backend language. **Go is a bonus**: it's our main language.

Handover

Please hand over the assignment through **GitHub**.

Bonuses

- Write it in **Go**
- Add metrics for throttle rate, error rates, and request counts per buyer
- An HTTP endpoint `GET /state` that returns the current throttle state for all buyers:

```
{
  "buyers": [
    { "buyer_id": "buyer_123", "throttle_rate": 0.95, "reason": "failure_rate" },
    { "buyer_id": "buyer_456", "throttle_rate": 0.35, "reason": "recovering" },
    { "buyer_id": "buyer_789", "throttle_rate": 0.0,  "reason": "" }
  ]
}

```

- Anything else you think makes the system better: tell us why in the README.

Good luck, and feel free to ask any questions :)